VDB

FOR RAG SYSTEMS

After creating Embedding you need somewhere to store chunk text, metadata, embedding vector

Examples :

{

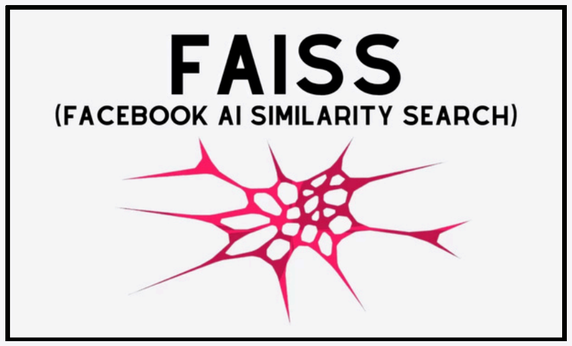
“text” : “Transformers were introduced in 2017”,

“embedding” : [0.12, 0.44, ...],

“metadata” “ { “source” : “paper.pdf”,

“page” : 10 }

}


FAISS
(FACEBOOK AI SIMILARITY SEARCH)




drant


milvus


Pinecone


elasticsearch


redis

id	embedding	chunk	metadata
1	[0.12 , -0.85 , 0.9 , ...]	Microwave engineering is subfield of Teleco.....	{ }
2	[0.9 , -0.32 , -0.01 , ...]	Bert is enconder only model which focus	{ }
3	[-0.912, -0.311, 0.88, ...]	Difference between RNN and LSTM is	{ }

what is difference between vector store and VDB ?

vector store is a system that can store vectors, retrieve vectors, search similar vectors
= storage + similarity search

but wait, what if computer shuts down, what if 500 employees want to access it, what if we have 20 billion vectors ??
in this case we need multiple servers, we need replication, we need networking, we need user authentication, we need backups, we need cloud deployment also.
now we are no longer talking about a simple vector store, Now we need a Vector Database
Vector Database is a Vector store + Everything needed for production

Vector store

Storage
id · embedding · chunk · metadata

Similarity search
ANN · cosine · dot product

FAISS · Annoy · HNSWlib

is part of

Vector database

Vector store
storage + similarity search

Persistence
survives restarts

Filtering
by metadata

Authentication
access control

REST API
HTTP endpoints

Replication
high availability

Sharding
distributed splits

Backups
disaster recovery

Scaling
horizontal + cloud

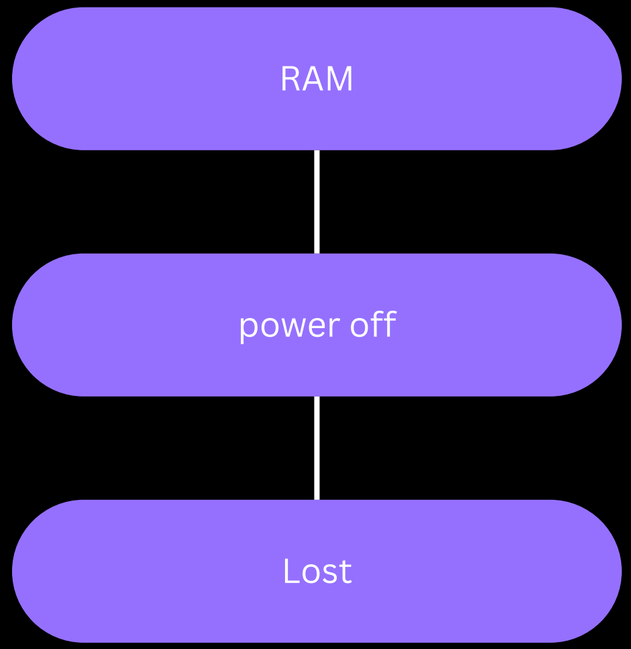
Metadata indexing
fast structured filtering alongside vectors

Qdrant · Pinecone · Weaviate · Milvus

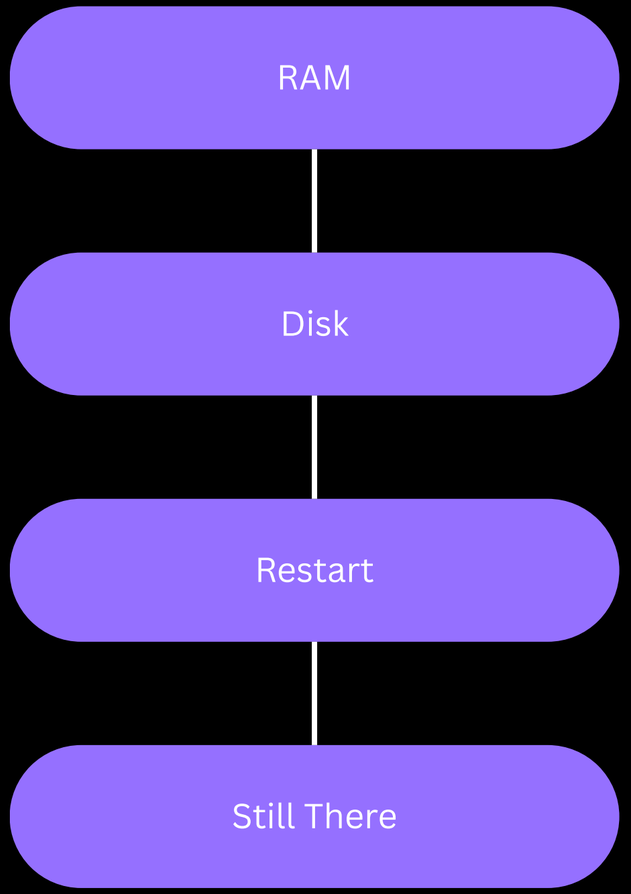
like FAISS : PostgreSQL
same core idea — totally different production readiness

Persistence

Data survives after shutdown

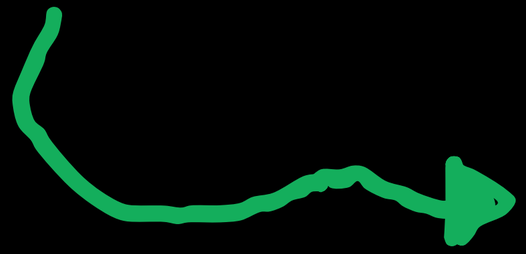


without Persistence



With Persistence

FAISS	Qdrant	Pinecone
Persistence is None by default, Everything lives in RAM, when your program stops, data is gone, you have to manually call Faiss.write_index( ) to save to disk, and reload it everytime	Automatic, Data is written to disk continuously	Fully Managed Cloud Persistance, Data Lives on pinecone’s Server



In real RAG system, you don’t want to re-embed 1 mililon documents every time your app restarts, that’s why FAISS is only good for prototyping or small experiments

Filtering

say you have 1 million documents from 10 different companies stored in one vector database, A user from company A Searches for Something, But you don’t want results from Company B,C,D, you need to restrict the search to only company A’s documents
Pure similarity search can’t do this, it just finds nearest vectors regardless of who they belong to

Each vector has metadata stored alongside it :

```
{ id : 1042,
embedding : [ 0.12 , -0.85 , ..... ],
chunk  : “how to reset your router”,
metadata : {
                company : “company_a “,
                category : “networking”,
                language : “en”,
                date : “2025-01-15”
}
}
```

when you Query you add a filter condition:

```
results = client.search(
            collection = “Docs”,
            query_filter = {
           “Company” : “company_a”,
           “language” : “en”
            }
)
```

Now it only searches vectors where company = company_a
and language = en

Two strategies DBs use internally :

[1] pre-filtering

- filter first, then Run ANN
- good when filter removes most data

[2] Post filtering

- Run ANN first, then filter
- risk, because it might return fewer results than requested

Indexing

happens before users search, if prepares the vectors so searching can be fast, it organizing vectors in a smart way so similarity search is fast, instead of computing similarity between Query to every Vector One by one

without Indexing

Query Vector → similarity(Query, V1)

→ similarity(Query, V2)

→ similarity(Query, V3)

→ similarity(Query, V4)

→ similarity(Query, V5)

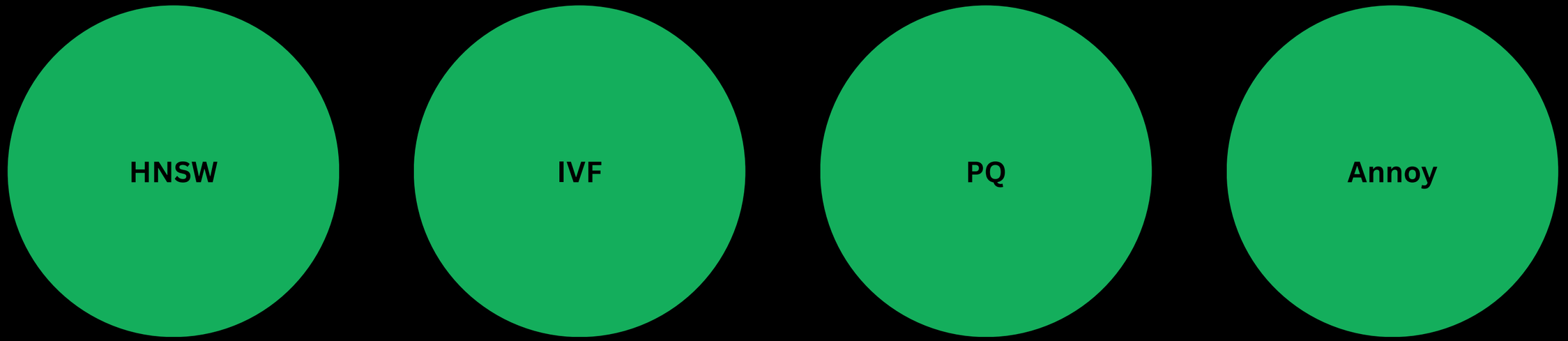
→ similarity(Query, V6)

.....

→ similarity(Query, V1000000)

This is called exact search
it is accurate but impossibly slow at scale

Main Indexing Algorithms



Searching

Indexing happens before users search, but searching happens after a user asks a Question, it uses the index to find the nearest vector

ANN

Indexing is preprocessing

Omar Ahmed

Thank you